A CASUAL-READING FIELD GUIDE

AI Infrastructure Engineering

The 12-stage roadmap, fact-checked and expanded: the physics behind each stage, the napkin math, the claims that are overstated, and practice projects that actually teach the right lesson.

Written for an experienced systems engineer. October 2026. Hardware numbers are vendor peak specs and are rounded; treat them as napkin-math inputs, not guarantees.

CONTENTS

What's inside

Read it front to back, or jump to a stage. Every stage follows the same shape: the idea, the napkin math, a myth check, a better practice project, and where to read more.

0	How to read this guide	The one mental model that ties everything together
1	Systems & GPU foundations	SMs, warps, memory hierarchy, interconnects
2	Transformer inference physics	Prefill vs decode, the roofline, KV-cache math
3	Serving engines & batching	vLLM, SGLang, PagedAttention, chunked prefill
4	KV-cache & memory optimization	Prefix caching, offload, cache-aware routing
5	Quantization & compression	FP8, INT4, AWQ, GPTQ, and how to measure damage
6	Kernel-level engineering	Triton, FlashAttention, fusion, CUDA graphs
7	Distributed inference & parallelism	TP, PP, EP and where each belongs
8	Speculative decoding	Draft models, EAGLE/Medusa, acceptance math
9	Multi-node & interconnects	NCCL, RDMA, InfiniBand, disaggregated serving
10	Cluster orchestration & scheduling	Kubernetes, Ray, Slurm, MIG, autoscaling
11	Gateways, routing & observability	SLOs, semantic routing, DCGM, OpenTelemetry
12	Public benchmarks & teardowns	Methodology that survives scrutiny
13	Beyond the roadmap	Multi-LoRA, structured output, non-NVIDIA, batch, reliability
A	A compressed 6-month plan	For someone with 20 years of systems experience
B	Cheat sheets	Hardware numbers, formulas, glossary, reading list

CHAPTER 0

How to read this guide

The original roadmap is a good list of topics. What it lacks is the thread that connects them. Here is that thread.

The one idea: everything is a fight over bytes

A GPU can do arithmetic far faster than it can fetch the numbers to do arithmetic on. An H100 can perform roughly 1,000 trillion BF16 multiply-adds-worth of FLOPs per second, but can only read about 3.35 TB per second from its main memory. Divide one by the other and you get roughly **300 FLOPs per byte**. Unless your workload does about 300 operations on every byte it loads, the compute units sit idle waiting for memory.

Almost every stage in this roadmap is a different answer to that single problem:

- **Batching** (Stage 3) reuses each weight byte across many requests, raising the FLOPs-per-byte ratio.
- **KV-cache tricks** (Stage 4) avoid recomputing or re-reading bytes you already have.
- **Quantization** (Stage 5) makes each number fewer bytes.
- **Kernel fusion** (Stage 6) keeps intermediate results in fast on-chip memory instead of round-tripping to main memory.
- **Parallelism** (Stage 7) spreads the bytes over more GPUs, then pays a new tax: moving bytes between them.
- **Speculative decoding** (Stage 8) gets several tokens out of one pass over the weights.
- **Interconnects and disaggregation** (Stage 9) are about moving bytes between machines cheaply.

Stages 10–12 are the business layer: keeping expensive hardware busy, routing work to the cheapest thing that meets the quality bar, and proving your numbers in public.

The shape of each chapter

NAPKIN MATH

Back-of-envelope numbers you should be able to do in your head during a design review.

MYTH CHECK

Where the original roadmap overstates, oversimplifies, or is simply wrong.

PRACTICE, DONE RIGHT

The roadmap's exercise, corrected where needed, with what to measure and what "done" looks like.

READ MORE

The handful of papers, posts and docs worth your time. Not an exhaustive list.

A word on the hype

The original post opens with "you will never be unemployed again." Nobody can promise that, and it isn't needed to justify the field. The real case is simpler: inference cost is now the largest line item for anyone running models at scale, and the people who can cut it by 2–5x without hurting quality are scarce. That is a durable, well-paid skill, and it builds on exactly the systems intuition you already have.

STAGE 1

Systems & GPU foundations

You can't reason about inference performance until you can picture where a byte lives and how long it takes to get to an arithmetic unit.

Time: 3–5 weeks	**Hardware:** any NVIDIA GPU; one rented H100 for the final comparison	**Languages:** C++, CUDA, Python

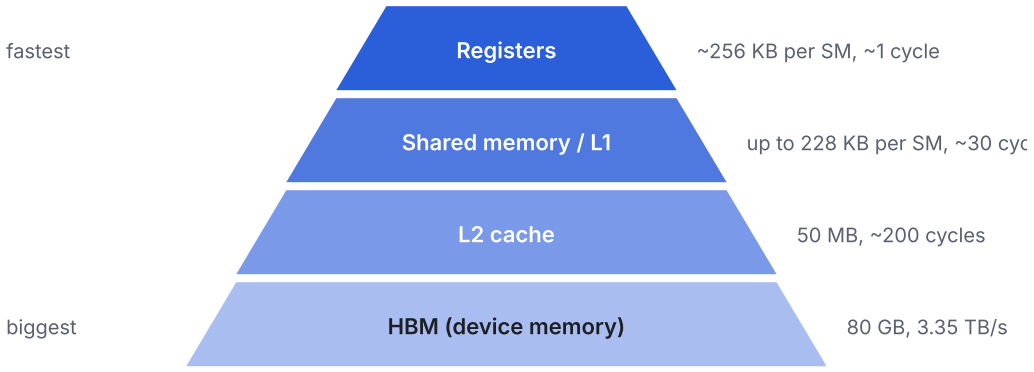
The GPU as a CPU person should picture it

A GPU is a collection of **Streaming Multiprocessors (SMs)**. An H100 SXM has 132 of them. Each SM is roughly a very wide, very simple core with its own register file and a small, fast scratchpad (shared memory). The SM runs threads in groups of 32 called **warps**. All 32 threads in a warp execute the same instruction at the same time (SIMT). If threads in a warp take different branches, the warp runs both paths with some threads masked off. That's *warp divergence*, and it's the GPU equivalent of a branch mispredict that never ends.

You write a kernel for one thread, then launch a **grid** of **thread blocks**. A block (up to 1,024 threads) is scheduled onto exactly one SM and stays there; its threads can share memory and synchronize. Blocks cannot talk to each other cheaply (Hopper's thread-block clusters loosen this a little).

The key trick is **latency hiding by oversubscription**. A CPU fights memory latency with big caches and out-of-order execution. A GPU instead keeps many warps resident per SM and switches between them every cycle at zero cost. When one warp stalls on a memory load, the scheduler issues from another. That's why *occupancy* (how many warps are resident) matters, and why it's limited by registers and shared memory per thread.

The memory hierarchy



H100 SXM memory hierarchy, approximate. Latencies are order-of-magnitude.

Two rules fall out of this picture and drive nearly every kernel you'll ever write:

1. **Coalesce your global memory accesses.** When the 32 threads of a warp read 32 consecutive 4-byte values, the hardware merges them into a few wide transactions. If they read scattered addresses, you pay for many transactions and use a fraction of each.
2. **Reuse data from shared memory and registers.** Load a tile from HBM once, then let many threads use it many times from on-chip memory. This is the whole idea behind tiled matmul and FlashAttention.

Interconnects: PCIe vs NVLink

Link	Bandwidth (approx.)	Where it matters
HBM3 on H100	3.35 TB/s	Every kernel
NVLink 4 (H100, per GPU)	900 GB/s total	Tensor parallelism inside a node
PCIe Gen5 ×16	~64 GB/s each way	Host↔GPU copies, CPU KV offload, PCIe-only cards
InfiniBand NDR, per port	400 Gb/s ≈ 50 GB/s	Between nodes (usually 8 ports per node)

The gap between HBM and NVLink is ~4x. The gap between NVLink and a single InfiniBand port is ~18x. Keep those ratios in your head; they decide how you shard models in Stage 7.

> MYTH CHECK
>
> **"Learn C++ *and* Rust."** Rust is a fine language and shows up in some serving infrastructure (routers, tokenizers, gateways), but it's not on the critical path for GPU work. The kernel path is C++ → CUDA → Triton. Learn Rust later if a job needs it.
>
> **"You must understand the silicon."** Partly true. You need a working model of SMs, warps, the memory hierarchy and tensor cores. You don't need to know transistor-level details or memorize the ISA.

PRACTICE, DONE RIGHT

Write SGEMM from scratch and climb the ladder against cuBLAS. This is the classic exercise and the roadmap got it right. Do it in steps and measure each one:

1. Naive kernel: one thread per output element.
2. Fix coalescing (just change the index mapping; expect a big jump).
3. Shared-memory tiling.
4. Register tiling: each thread computes a small block of outputs (1D, then 2D).
5. Vectorized loads (`float4`), then warp-level tiling.
6. Stretch goal: use tensor cores (WMMA/MMA) in BF16 and compare against cuBLAS in BF16.

Measure achieved GFLOP/s and % of cuBLAS at 4096×4096. Use `ncu` (Nsight Compute) to check memory throughput, occupancy and warp stall reasons at each step. **Done** means 70–90% of cuBLAS for FP32 and that you can explain, with profiler evidence, what each step fixed.

READ MORE

- Simon Boehm, *How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance*. The step-by-step version of the exercise above.
- Hwu, Kirk & El Hajj, *Programming Massively Parallel Processors* (4th ed.). The textbook.
- GPU MODE lecture series and Discord. The best free community for this material.
- NVIDIA CUDA C++ Programming Guide and Best Practices Guide.

STAGE 2

Transformer inference physics

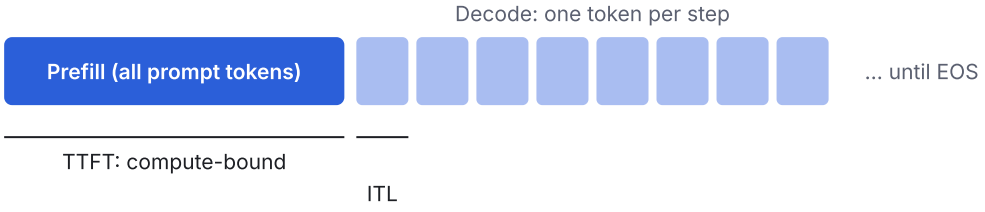
The most important chapter. If you understand this one, the rest of the roadmap is engineering.

Time: 2–3 weeks **Hardware:** one GPU that fits an 8B model **Tools:** Nsight Systems, PyTorch profiler

Two phases with opposite personalities

Generating a response happens in two phases:

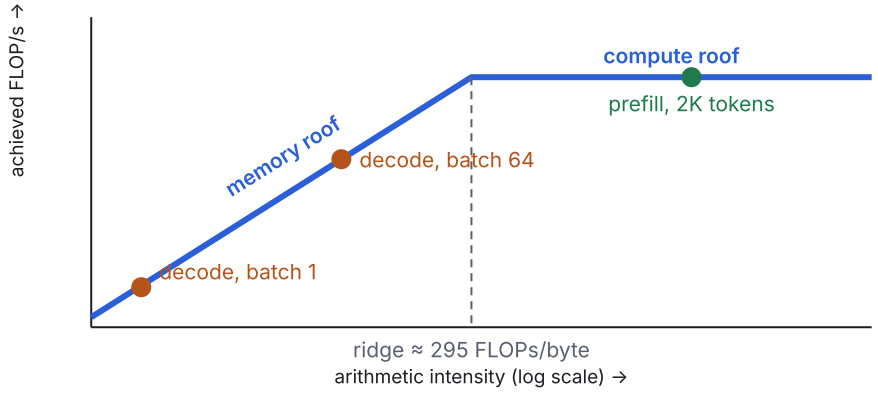
- **Prefill:** the model processes the whole prompt at once. With 2,000 prompt tokens, every weight matrix is multiplied by a 2,000-row matrix. That's a real matrix-matrix multiply with lots of reuse per weight byte. **Prefill is compute-bound.** It determines Time To First Token (TTFT).
- **Decode:** the model generates one token at a time. For each new token, every weight is read from HBM to multiply it by a single vector. Almost no reuse. **Decode at small batch sizes is memory-bandwidth-bound.** It determines the time between tokens (Inter-Token Latency, ITL, also called TPOT).



One request's timeline. Prefill is one big pass; decode is many small passes that each re-read all the weights.

Arithmetic intensity and the roofline

Arithmetic intensity = FLOPs performed ÷ bytes moved from memory. The **roofline model** says your achievable throughput is the lower of two ceilings: peak compute, or memory bandwidth × arithmetic intensity. The crossover point, the *ridge point*, is peak FLOPs ÷ bandwidth. For an H100 in BF16 that's about 989 TFLOP/s ÷ 3.35 TB/s ≈ **295 FLOPs/byte**.



Roofline for an H100 (BF16). Decode climbs the slope as batch size grows; prefill sits on the flat roof.

NAPKIN MATH: DECODE INTENSITY ≈ BATCH SIZE

A weight matrix of BF16 values costs 2 bytes per weight to read. For each request in the batch, each weight does one multiply and one add: 2 FLOPs. So with batch size B, intensity ≈ $2B \div 2 = B$ **FLOPs/byte**.

Batch 1 → intensity 1. You're using about 0.3% of the H100's compute. To reach the ridge you'd need a batch of ~300 sequences, and before you get there you usually run out of KV-cache memory or blow your latency SLO. That's why decode is "mostly memory-bound" in practice.

NAPKIN MATH: THE DECODE SPEED LIMIT

At batch 1, every token requires reading every weight once. So:

max tokens/s ≈ memory bandwidth ÷ model size in bytes

Llama 3 8B in BF16 is ~16 GB. On an H100: 3.35 TB/s ÷ 16 GB ≈ **210 tokens/s** upper bound. Real systems get maybe 60–80% of that. If someone claims 500 tok/s single-stream on an H100 for an unquantized 8B model without speculative decoding, something is off.

KV-cache math

Attention needs the Key and Value vectors of every previous token. Recomputing them each step would be quadratic, so we cache them. The cost:

```
KV bytes per token = 2 (K and V) × layers × kv_heads × head_dim × bytes_per_value
```

Model (BF16)	Layers	KV heads	head_dim	Per token	128K-token sequence
Llama 3 8B	32	8	128	128 KB	~16 GB
Llama 3 70B	80	8	128	320 KB	~40 GB

Note the "KV heads = 8". These models use **Grouped-Query Attention (GQA)**: 64 query heads share 8 KV heads. Without GQA the 70B model would need 8x more KV memory. Architectures like DeepSeek's Multi-head Latent Attention (MLA) compress it further. **Model architecture is a serving decision.**

The KV cache also means decode is *doubly* memory-bound: each step reads the weights *and* the whole KV cache of every sequence in the batch. At long contexts, KV reads can exceed weight reads. That's why attention kernels and KV compression matter so much.

MYTH CHECK

"LLM inference is almost always memory-bound, not compute-bound." True for decode at small to moderate batch sizes, and that's where most latency-sensitive serving lives. But prefill is compute-bound, large-batch decode moves toward compute-bound for the weight GEMMs, and real servers mix both phases in the same batch. The skill isn't memorizing "memory-bound." It's being able to place any given workload on the roofline.

PRACTICE, DONE RIGHT

Load Llama 3.1 8B (or similar) in HuggingFace Transformers. Profile with `nsys profile`:

1. Run one 2,000-token prefill and 200 decode steps at batch 1. In the timeline, find the GEMMs. Prefill GEMMs should be long and few; decode should be hundreds of tiny kernels with gaps between them (that's Python and launch overhead, a preview of Stage 6).
2. For one decode step, compute achieved bandwidth: bytes read (weights + KV) ÷ step time. Compare to 3.35 TB/s.
3. Repeat decode at batch 1, 8, 32, 128. Plot tokens/s and per-step latency against batch size. You should see throughput rise nearly linearly while latency stays flat, then bend. That bend is the roofline in your own data.

Done means you can predict step latency within ~30% from napkin math before running.

READ MORE

- Horace He, *Making Deep Learning Go Brrrr From First Principles*. Compute vs memory vs overhead, beautifully explained.
- Google DeepMind, *How to Scale Your Model* (the "scaling book"). Rooflines, sharding and inference math. Written with TPUs in mind but the ideas carry over directly.
- Kipply, *Transformer Inference Arithmetic*. The classic napkin-math post.

STAGE 3

Modern serving engines & batching

Batching is how you climb the roofline. Serving engines exist to batch aggressively without wasting memory or wrecking latency.

Time: 3–4 weeks **Hardware:** 1 GPU for 8B; 2×H100 or more for 70B **Software:** vLLM, SGLang

From static to continuous batching

Static batching collects N requests, runs them together, and waits for the longest one to finish before starting the next batch. Short requests sit idle, and new arrivals wait. **Continuous batching** (introduced by the Orca paper as "iteration-level scheduling") reschedules every decode step: finished sequences leave, new ones join immediately. This alone typically raises throughput several-fold on realistic traffic.

PagedAttention: virtual memory for the KV cache

Early servers reserved a contiguous KV buffer per request sized for its maximum possible length. Most requests never use that much, and the leftover gaps fragment memory. The vLLM paper measured that existing systems wasted **60–80% of KV-cache memory** this way.

PagedAttention borrows the OS idea of paging. The KV cache is split into fixed-size blocks (16 tokens per block by default in vLLM). Each sequence has a block table mapping logical positions to physical blocks, allocated on demand. Waste drops to under 4% (only the last partial block). Bonus: blocks can be shared between sequences copy-on-write, which makes parallel sampling and prefix sharing almost free.

As an OS-literate engineer, you'll find this chapter comfortable: it's a page table, a free list, and an eviction policy.

SGLang and RadixAttention

SGLang keeps the KV cache in a **radix tree** keyed by token sequences. Any new request that shares a prefix with something in the tree (a system prompt, few-shot examples, earlier turns of a chat) reuses those KV blocks automatically, with LRU eviction. vLLM has equivalent automatic prefix caching (hash-based blocks). In 2026 the two engines are close on most workloads; differences show up on specific patterns (structured output, heavy prefix sharing, specific models and kernels). That's why you benchmark (Stage 12).

Chunked prefill: protecting decode from long prompts

A 32K-token prefill can take hundreds of milliseconds. If the scheduler runs it as one step, every decoding request in the batch stalls for that long, and your ITL p99 spikes. **Chunked prefill** (from the Sarathi-Serve work) splits the long prefill into chunks, e.g. 512–2,048 tokens, and mixes one

chunk into each step alongside the ongoing decodes. Decode keeps flowing; the long prompt finishes a bit later.

The tuning knob is the token budget per step (in vLLM, `max_num_batched_tokens`):

- **Smaller budget** → smoother ITL for everyone, slower TTFT for long prompts, slightly lower peak throughput.
- **Larger budget** → better prefill efficiency and TTFT, worse ITL tail.

NAPKIN MATH: WHAT FITS ON THE GPU

Llama 3 70B in BF16 = ~140 GB of weights. That needs at least 2×H100 (160 GB), which leaves only ~10–15 GB for KV cache after activations and overhead: about 30K–45K tokens total across *all* concurrent requests at 320 KB/token. That's barely one long-context user. In practice you run 70B on 4× or 8×H100, or quantize to FP8 (Stage 5) so it fits comfortably on 2.

MYTH CHECK

"Naive batching leaves 60% of GPU VRAM wasted." The real figure is 60–80% of **KV-cache** memory wasted by contiguous pre-allocation, not 60% of total VRAM. Still a big deal; just be precise in interviews. Engineers who quote the exact claim get more trust.

PRACTICE, DONE RIGHT

Deploy a 70B model and tune chunked prefill against an SLO.

1. Serve Llama 3.x 70B on 4×H100 with tensor parallelism 4 (or FP8 on 2×H100).
2. Build a mixed workload: 80% chat requests (~1K in / 300 out) plus 20% long-context requests (16K–32K in). Use Poisson arrivals, not a fixed concurrency.
3. Set an SLO up front, for example: TTFT p95 < 2 s, ITL p99 < 80 ms.
4. Sweep `max_num_batched_tokens` (512, 1K, 2K, 4K, 8K) and request rate. For each setting, find the maximum request rate that still meets the SLO ("goodput").

Done means a plot of goodput vs chunk budget, with a one-paragraph explanation of why the best setting wins.

READ MORE

- Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention* (SOSP 2023), the vLLM paper.
- Yu et al., *Orca* (OSDI 2022), continuous batching.
- Agrawal et al., *Sarathi-Serve* (OSDI 2024), chunked prefill.
- Zheng et al., *SGLang* (RadixAttention).

STAGE 4

KV-cache & memory optimization

The cheapest token is the one you don't have to recompute. This stage is about treating the KV cache as a first-class, tiered, shareable resource.

Time: 2–3 weeks	**Hardware:** 2+ GPU replicas (can be small models)	**Software:** vLLM/SGLang, a proxy you write, LMCache

The four levers

1. Prefix caching

If two requests start with the same 4,000-token system prompt, the KV for those tokens is identical. Compute it once, reuse it. The second request skips that part of prefill, so its TTFT drops by roughly the share of the prompt that's cached. Both vLLM and SGLang do this automatically within one replica.

2. Multi-turn reuse

In chat, turn N's prompt is turn N−1's prompt plus the response plus a new message. If the KV for turn N−1 is still in cache, you only prefill the new part. Without reuse, a 20-turn conversation re-prefills the entire history 20 times.

3. KV quantization

Store K and V in FP8 instead of BF16: half the memory, so roughly twice the tokens in cache and twice the concurrent sequences. It also halves KV read bandwidth during decode. Quality impact is usually small but must be checked, especially on long contexts.

4. Tiered offload

When GPU memory fills, evicted KV blocks can go to CPU RAM (or local NVMe, or a shared store) instead of being thrown away. Reloading over PCIe Gen5 (~64 GB/s) is usually much faster than recomputing a long prefix. LMCache and NVIDIA Dynamo's KV block manager implement this; Mooncake (from Moonshot AI) built a whole architecture around a distributed KV store.

> **NAPKIN MATH: RELOAD VS RECOMPUTE**
>
> A 32K-token prefix on Llama 3 70B = 32K × 320 KB ≈ 10 GB of KV. Over PCIe Gen5 at ~50 GB/s effective: ~0.2 s. Recomputing that prefill on 4×H100 takes roughly 1–3 s depending on the stack. Reload wins by an order of magnitude. For a 500-token prefix it's a wash, and you shouldn't bother.

The routing problem

Prefix caching works per replica. With 8 replicas behind a round-robin load balancer, a given system prompt lands on a random replica, so your cache hit rate drops to roughly 1/8 of what it could be. The fix is **cache-aware routing**: send requests that share a prefix to the same replica.

But pure affinity creates hotspots: one popular system prompt can overload a single replica. Good routers balance two signals: **expected cache hit** (prefix match length) and **load** (queue depth, KV usage). A common approach is consistent hashing on a prefix hash, with spillover when a replica's load exceeds a threshold.

MYTH CHECK

"Reused cache can slash TTFT by 80%." On the right workload, yes, and sometimes more: long shared system prompts, RAG over a fixed document set, agents re-sending long histories. For workloads with unique prompts, the gain is close to zero. Always state the hit rate and the cached share of the prompt alongside the TTFT claim.

"Build a routing proxy." Good exercise, but know the prior art: the SGLang router, the vLLM production-stack router, llm-d's inference scheduler, and NVIDIA Dynamo's KV-aware router all do this. Benchmark your router against one of them.

PRACTICE, DONE RIGHT

1. Run 4 replicas of a small model (an 8B on 4 GPUs, or even a 1B on fewer).
2. Generate traffic with ~20 distinct system prompts (2K–4K tokens each), with Zipf-distributed popularity, plus short unique user messages.
3. Implement three routing policies in a small proxy: round-robin, pure prefix-hash affinity, and prefix-hash with load-based spillover.
4. Measure prefix cache hit rate (both engines expose it as a metric), TTFT p50/p95, and per-replica load imbalance.

Done means you can show that pure affinity beats round-robin on TTFT but loses on p99 under skew, and that the spillover version gets the best of both.

READ MORE

- vLLM docs: Automatic Prefix Caching. SGLang docs: RadixAttention and the router.
- LMCache project. Mooncake paper (KVCache-centric disaggregated architecture).
- llm-d and NVIDIA Dynamo documentation on KV-aware routing.

STAGE 5

Quantization & compression

Fewer bytes per weight means faster decode, more room for KV cache, and smaller hardware bills. The catch is quality, and measuring it honestly is the hard part.

Time: 2–3 weeks | **Hardware:** Hopper or newer for native FP8 | **Software:** llm-compressor, TensorRT Model Optimizer, lm-evaluation-harness

The menu

Format	What gets quantized	Typical quality cost	Notes
FP8 (W8A8)	Weights and activations	Usually near-lossless	Native tensor-core support on Hopper/Blackwell. The default first step for production.
INT8 W8A8 (SmoothQuant)	Weights and activations	Small	Good on Ampere, which lacks FP8. Needs outlier handling.
INT4 weight-only (AWQ, GPTQ)	Weights only (W4A16)	Small to noticeable	Great for memory-bound decode at small batch sizes. Activations stay 16-bit.
FP4 (NVFP4, MXFP4)	Weights, optionally activations	Varies; needs care	Native on Blackwell. Becoming common for frontier models in 2026.
2:4 sparsity	Half of weights zeroed in a fixed pattern	Needs fine-tuning to recover	Hardware-supported but less widely used than quantization.

How the main methods differ

- **GPTQ** quantizes weights one column at a time and adjusts the remaining columns to compensate for the error, using second-order (Hessian) information from a small calibration set.
- **AWQ** (Activation-aware Weight Quantization) observes that a small fraction of weight channels matter most, the ones that see large activations. It scales those channels up before quantizing so they keep more precision.
- **SmoothQuant** moves the difficulty of quantizing activations (which have outliers) into the weights (which are easy to quantize) with a mathematically equivalent per-channel scaling.
- **Calibration** (as in TensorRT-LLM / ModelOpt) means running representative data through the model to choose scaling factors. Calibration data that doesn't match your traffic is a classic source of silent quality loss.

NAPKIN MATH: WHAT QUANTIZATION BUYS

Llama 3 70B	Weights	Min. H100s	Decode limit, batch 1, 1×H100-equivalent bandwidth
BF16	140 GB	2	~24 tok/s
FP8	70 GB	1 (tight)	~48 tok/s
INT4 (W4A16)	~38 GB	1	~88 tok/s

The last column is just 3.35 TB/s ÷ weight bytes. Halving bytes roughly doubles the memory-bound decode ceiling. At high batch sizes, the gain from weight-only INT4 shrinks, because the workload becomes compute-bound and the 16-bit activations still do 16-bit math.

MYTH CHECK

"Benchmark the perplexity drop." Perplexity is a fine smoke test but a weak quality signal. Quantized models can keep perplexity within 1% while losing several points on math, code or long-context retrieval. Use task evals: GSM8K or MATH for reasoning, HumanEval-style for code, a long-context retrieval test (e.g. RULER or needle-in-a-haystack), and most important, **your own golden set** from real traffic, scored against the BF16 model's outputs.

"Quantization is the only way to fit frontier models on edge GPUs." It's the main way, alongside distillation into smaller models and pruning. "Only" is too strong.

PRACTICE, DONE RIGHT

1. Take an 8B model. Produce FP8 (dynamic or calibrated), AWQ INT4 and GPTQ INT4 versions with llm-compressor or ModelOpt.
2. Serve each in vLLM. Measure memory, max KV capacity, single-stream tokens/s, and throughput at a fixed latency SLO.
3. Evaluate with lm-evaluation-harness on 3–4 tasks, plus a 200-prompt golden set compared against BF16 outputs (exact match or an LLM judge).
4. Try a deliberately mismatched calibration set (e.g. calibrate on Wikipedia, evaluate on code) and see what happens.

Done means a table of latency, throughput, memory and quality for each variant, and a recommendation you'd defend.

READ MORE

- Frantar et al., *GPTQ*. Lin et al., *AWQ*. Xiao et al., *SmoothQuant*.
- Micikevicius et al., *FP8 Formats for Deep Learning*.
- vLLM's llm-compressor docs; NVIDIA TensorRT Model Optimizer docs.

STAGE 6

Kernel-level engineering

Where you stop using the framework's kernels and start writing your own. Triton makes this accessible; FlashAttention shows what's possible.

Time: 3–4 weeks **Hardware:** one modern GPU **Software:** Triton, PyTorch, Nsight Compute

Why fusion matters

Consider RMSNorm in plain PyTorch: square, mean, add epsilon, rsqrt, multiply, multiply by weight. Each operation is a separate kernel that reads its input from HBM and writes its output back. That's 5–6 full round trips through memory for an operation that only needs one read and one write. Every one of those ops is memory-bound with tiny arithmetic intensity. **Fusing** them into one kernel keeps intermediates in registers and can be 3–6x faster for this op.

Triton in one paragraph

Triton is a Python-embedded language where you write kernels at the level of *blocks* of data rather than individual threads. You say "load this block of 1,024 elements, compute, store," and the compiler handles thread mapping, memory coalescing, and shared memory. You give up some control compared to CUDA, and get 80–100% of hand-tuned performance for most memory-bound ops in a fraction of the code. `torch.compile` generates Triton kernels under the hood, so reading its output is a great way to learn.

FlashAttention: fusion applied to attention

Naive attention computes the full N×N score matrix, writes it to HBM, applies softmax, writes again, then multiplies by V. At long sequence lengths that matrix is enormous. **FlashAttention** tiles Q, K and V into blocks that fit in shared memory, and uses the *online softmax* trick (keeping a running max and running sum) so it never materializes the full matrix. Same exact result, far less memory traffic. FlashAttention-2 improved work partitioning; FlashAttention-3 uses Hopper-specific features (asynchronous TMA copies, warp specialization, FP8). For decode, specialized kernels (FlashDecoding, FlashInfer) split the KV sequence across blocks so a single query can use the whole GPU.

CUDA graphs: killing launch overhead

A decode step for an 8B model can launch hundreds of small kernels. Each launch has CPU-side overhead of a few microseconds, and Python adds more. When the GPU work per kernel is also only a few microseconds, the GPU sits idle between launches (you saw these gaps in Stage 2's Nsight trace). **CUDA graphs** record the whole sequence of launches once and replay it with a single call. Serving engines capture graphs for a set of batch sizes and pad to the nearest one. That's the source of vLLM's startup "capturing CUDA graphs" phase.

MYTH CHECK

"Python overhead kills inference." At small batch sizes in eager PyTorch, yes, it dominates decode. In a production engine with CUDA graphs and fused kernels, Python is mostly off the hot path. The real lesson: measure where time goes (GPU busy vs gaps) before optimizing.

PRACTICE, DONE RIGHT

1. Write a fused RMSNorm forward kernel in Triton (one program per row, load, compute, normalize, scale, store).
2. Benchmark against eager PyTorch, `torch.compile`, and an existing fused version (Liger Kernel or the vLLM kernel) across hidden sizes 2K–16K and token counts 1–8K.
3. Report achieved memory bandwidth as a percentage of peak. A good memory-bound kernel reaches 80–90%.
4. Stretch: fuse the residual add into the same kernel (`x = x + residual; y = rmsnorm(x)`), which is what real engines do. Then write a Triton softmax and compare with the online-softmax variant.

Done means a bandwidth-vs-size plot and an explanation of where your kernel loses to the best one.

READ MORE

- Official Triton tutorials (vector add → fused softmax → matmul → layer norm).
- Dao et al., *FlashAttention* 1, 2 and 3 papers.
- FlashInfer project and paper; Liger Kernel repository for production Triton examples.

STAGE 7

Distributed inference & parallelism

When a model doesn't fit on one GPU, you split it. Each way of splitting trades memory for a different kind of communication.

Time: 2–3 weeks	**Hardware:** one 8-GPU node; two nodes for the cross-node part	**Software:** vLLM / SGLang / TensorRT-LLM

The three main strategies

Tensor parallelism (TP)

Split each weight matrix across GPUs. In the Megatron layout, the first MLP matrix is split by columns and the second by rows, so each GPU does its share and the results are combined with one **all-reduce**. Same for attention (split by heads). That's **two all-reduces per transformer layer, every forward pass**. TP cuts per-GPU memory and per-token latency, but it's chatty: it needs NVLink-class bandwidth and low latency. Keep TP **inside a node**.

Pipeline parallelism (PP)

Split the model by layers: GPUs 0–3 hold layers 1–40, GPUs 4–7 hold 41–80. Only activations cross the boundary, once per stage, so communication is light and tolerates slower links. The cost is pipeline bubbles and added latency per stage. PP is the natural way to **cross nodes**.

Expert parallelism (EP)

For Mixture-of-Experts models (DeepSeek-V3, Qwen3 MoE, Llama 4, and others), each token is routed to a few of many experts. EP places different experts on different GPUs, so each token's hidden state must be sent to wherever its experts live and back: an **all-to-all** communication pattern. Load imbalance between experts is the main headache. Large-scale EP across nodes (as in DeepSeek's published inference setup) is one of the most demanding workloads in the field.

There's also **data parallelism** (whole replicas) and, for long contexts, **context/sequence parallelism**. Real deployments combine them: e.g. TP=8 within each node, PP=2 across nodes, multiple DP replicas behind a router.

NAPKIN MATH: DOES 405B NEED 8 NODES?

Llama 3.1 405B	Weights	Fits on
BF16	~810 GB	2× 8×H100 nodes, or 1× 8×H200 node (1,128 GB)
FP8 (Meta published an official FP8 version)	~405 GB	**1× 8×H100 node** (640 GB), with ~200 GB left for KV

Eight nodes is 64 GPUs: 6–8x more hardware than needed, and every tensor-parallel all-reduce would cross InfiniBand at ~1/18th of NVLink's bandwidth.

MYTH CHECK

"Shard a 405B model across 8 nodes and measure the communication overhead." This teaches the wrong lesson. Spreading tensor parallelism across nodes is the textbook mistake the exercise should help you avoid. It's also very expensive to rent 64 GPUs for a learning project.

"Single-GPU inference is dead for frontier models." True for frontier-scale dense and MoE models. But a large share of real production traffic runs on single-GPU models (7–32B, often quantized). Both worlds matter.

PRACTICE, DONE RIGHT

1. On one 8×H100 node, serve Llama 3.1 405B FP8 with TP=8. Record TTFT, ITL and throughput.
2. Serve a 70B model at TP=2, 4, 8 on the same node. Use Nsight Systems to measure the share of each decode step spent in NCCL all-reduce. Watch it grow as TP increases.
3. If you can get two nodes for a few hours: compare TP=16 across nodes against TP=8 + PP=2. The second should win clearly on decode latency.
4. Optional: serve a mid-size MoE model with EP and look at expert load imbalance.

Done means a chart of communication share vs parallelism layout, and a clear rule you'd apply in a design review.

READ MORE

- Shoeybi et al., *Megatron-LM*. The original tensor-parallel layout.
- *How to Scale Your Model*, the chapters on sharding and inference.
- DeepSeek-V3 technical report and their inference system write-ups (EP at scale).

STAGE 8

Speculative decoding

Decode is memory-bound, so the GPU has spare compute. Speculative decoding spends that spare compute to produce several tokens per pass over the weights.

Time: 2 weeks	**Hardware:** one node, draft and target on the same GPUs	**Software:** vLLM / SGLang speculative decoding, EAGLE

How it works

1. A cheap **draft** proposes *k* tokens (say 4).
2. The big **target** model runs once over all *k* positions in parallel. Because decode is memory-bound, checking 4 tokens costs about the same as generating 1.
3. A rejection-sampling rule accepts the longest prefix of draft tokens consistent with the target's distribution, then samples one more token from the target. You always get at least one token per pass.

With the standard acceptance rule, the output distribution is **mathematically identical** to sampling from the target alone. That's the "no quality cost" part, and it's genuinely true.

Kinds of drafters

- **Separate small model**, e.g. a 1B drafting for a 70B from the same family. Must share the tokenizer.
- **Medusa heads**: extra prediction heads attached to the target model that guess tokens +2, +3, ... in one pass.
- **EAGLE (EAGLE-2, EAGLE-3)**: a light draft head that reads the target's hidden states. Currently among the best-performing approaches and well supported in vLLM and SGLang.
- **N-gram / prompt lookup**: no model at all. Copies candidate continuations from the prompt. Excellent for code editing, summarization and RAG, where output repeats input.
- Some models now ship with built-in **multi-token prediction** heads (DeepSeek-V3's MTP) that double as drafters.

NAPKIN MATH: EXPECTED TOKENS PER PASS

If each draft token is accepted independently with probability α and you draft *k* tokens, the expected tokens per target pass is:

$$(1 - \alpha^{k+1}) \div (1 - \alpha)$$

α = 0.8, k = 4 → 3.36 tokens per pass. α = 0.6, k = 4 → 2.3. Subtract the drafter's own cost and the verification overhead, and you see why real speedups land around 1.5–3x, and why acceptance rate is the number to watch.

MYTH CHECK

"2x decode speed at zero quality cost." The quality part is right. The speed part depends on acceptance rate (which varies by task, often by a lot) and on **batch size**. At high batch sizes the GPU no longer has spare compute, so verification isn't free and the gain shrinks or disappears. Speculative decoding is a *latency* optimization for low-to-moderate load. Good engines turn it down or off as load rises.

"A local 3B model drafts tokens for a cloud 70B model." Each verify step would need a network round trip, typically 20–100 ms over the internet, while a decode step is roughly 10–30 ms. The network eats the entire gain. The draft must run next to the target, ideally on the same GPUs.

PRACTICE, DONE RIGHT

1. Serve a 70B target (or 8B, for cost) with three drafters: a small same-family model, an EAGLE head, and n-gram lookup.
2. Run three workloads: open-ended chat, code editing (output mostly copies input), and summarization.
3. Measure acceptance rate, mean accepted tokens per pass, single-stream tokens/s, and throughput at batch 1, 8, 32, 64.
4. Optional, if you want to keep the "local draft, cloud verify" idea: actually build it, add realistic network latency, and find the latency at which it stops helping. That's a nice short blog post.

Done means a speedup-vs-batch-size chart per drafter and workload, showing where each one stops paying off.

READ MORE

- Leviathan et al., *Fast Inference from Transformers via Speculative Decoding*; Chen et al., *Accelerating LLM Decoding with Speculative Sampling*.
- Cai et al., *Medusa*. Li et al., *EAGLE*, *EAGLE-2*, *EAGLE-3*.
- vLLM and SGLang docs on speculative decoding.

STAGE 9

Multi-node & hardware interconnects

Once a workload crosses machine boundaries, the network becomes part of the computer. This stage is about measuring it and designing around it.

Time: 2–3 weeks	**Hardware:** 2 nodes with InfiniBand or RoCE, for a few hours	**Software:** NCCL, nccl-tests, NIXL, Dynamo / llm-d

The stack, bottom to top

- **NVLink / NVSwitch**: GPU-to-GPU inside a node (900 GB/s per H100; NVL72 racks extend NVLink across 72 Blackwell GPUs).
- **RDMA**: Remote Direct Memory Access. The NIC reads and writes remote memory without involving either CPU. **GPUDirect RDMA** lets the NIC access GPU memory directly, skipping host memory entirely.
- **InfiniBand** (NDR 400 Gb/s, XDR 800 Gb/s) or **RoCE** (RDMA over Converged Ethernet). Training clusters typically give each GPU its own NIC.
- **NCCL**: NVIDIA's collective communication library. It implements all-reduce, all-gather, all-to-all and so on, picks ring or tree algorithms, and uses NVLink or the network as appropriate.
- **NIXL** and similar point-to-point libraries: built for moving KV-cache blocks between machines, which collectives don't fit well.

Disaggregated prefill and decode

Remember Stage 2: prefill is compute-bound and decode is memory-bound. Running them on the same GPU means they interfere: prefill bursts hurt decode latency, and you can't size hardware for either phase independently.

Disaggregation splits them: a **prefill pool** processes prompts and produces KV caches, which are transferred over RDMA to a **decode pool** that generates tokens. Each pool can use its own parallelism, batch size, and even hardware type, and you scale them independently based on traffic shape. DistServe and Splitwise made the academic case; Mooncake ran it at scale; NVIDIA Dynamo, llm-d, and native modes in vLLM and SGLang now make it deployable.

> **NAPKIN MATH: CAN THE NETWORK KEEP UP?**
>
> A 4K-token prompt on Llama 3 70B produces 4K × 320 KB ≈ 1.3 GB of KV. Over one 400 Gb/s port (~50 GB/s): ~26 ms. Over 8 ports in parallel: ~3 ms. Compared with a prefill time of a few hundred ms, the transfer is cheap, *if* you use RDMA and overlap it with compute (stream KV layer by layer as prefill progresses). Over plain TCP on 25 GbE it would be ~400 ms. That's why the transport matters.

MYTH CHECK

"Network latency is the new GPU bottleneck." For cross-node tensor parallelism and large-scale expert parallelism, yes. For data-parallel replicas or well-designed disaggregation, the network is usually not the limiter. Don't generalize.

"Disaggregation is the 2026 meta." It's the leading architecture for large deployments with long prompts and strict latency SLOs. For small deployments or short prompts, a co-located server with chunked prefill is simpler and often just as good. Disaggregation adds a KV transfer layer, two pools to balance, and more failure modes.

PRACTICE, DONE RIGHT

1. Run `nccl-tests` (`all_reduce_perf`, `alltoall_perf`) within one node and across two. Plot bus bandwidth vs message size. Note how small messages (typical for decode all-reduces) are latency-bound and far below peak.
2. Serve a 70B model with TP=8 in one node, then TP=16 across two. Compare decode ITL. Use `NCCL_DEBUG=INFO` to confirm which transport NCCL picked.
3. Deploy a disaggregated setup (1 prefill + 1 decode instance) with Dynamo, llm-d or vLLM's native support. Compare with a co-located baseline on a long-prompt workload: TTFT, ITL p99, and goodput at the same total GPU count.

Done means you can say, with your own data, when disaggregation is worth its complexity.

READ MORE

- Zhong et al., *DistServe* (OSDI 2024); Patel et al., *Splitwise*; Qin et al., *Mooncake*.
- NCCL documentation and the nccl-tests repository.
- NVIDIA Dynamo and llm-d architecture docs.

STAGE 10

Cluster orchestration & GPU scheduling

The most expensive GPU is the idle one. This stage is classic distributed-systems work with a very expensive resource at its center, and it's where your existing experience transfers most directly.

Time: 3–4 weeks	**Hardware:** a small k8s cluster with 2–4 GPU nodes (spot)	**Software:** Kubernetes, NVIDIA GPU Operator, KEDA, Ray / KubeRay

The landscape

Tool	What it's for
Kubernetes + NVIDIA GPU Operator	The default for serving. The operator installs drivers, the device plugin, DCGM exporter and MIG manager. Dynamic Resource Allocation (DRA) is the newer, more flexible way to request GPUs.
Ray / KubeRay	Python-native distributed compute. Ray Serve handles multi-model serving and autoscaling; vLLM uses Ray for multi-node setups.
Slurm	The HPC batch scheduler. Still dominant for training clusters and batch inference jobs.
Kueue / Volcano	Batch queueing and gang scheduling on Kubernetes (all-or-nothing placement for multi-GPU jobs).
KEDA	Event-driven autoscaling: scale on queue depth or any Prometheus metric instead of CPU.

GPU sharing

- **MIG (Multi-Instance GPU)**: hardware partitioning on A100/H100/B200, up to 7 isolated instances per GPU with dedicated memory and SMs. Great for small models and dev environments; fixed partition sizes.
- **Time-slicing**: multiple processes share a GPU in turns. No memory isolation.
- **MPS**: concurrent kernels from multiple processes. Better utilization, weaker isolation.

Why LLM autoscaling is harder than web autoscaling

- **Cold starts are slow.** Pulling a container, loading 140 GB of weights and capturing CUDA graphs can take several minutes. Mitigations: pre-pulled images, weights on local NVMe or a fast shared filesystem, model streaming loaders, and keeping a warm buffer.
- **CPU and GPU utilization are bad scaling signals.** Scale on queue depth, pending requests, KV-cache usage, or TTFT approaching its SLO.

- **Spot instances get reclaimed** with short notice. Draining must stop new requests, let in-flight ones finish (or migrate), and deregister from the router before the node disappears.

MYTH CHECK

"Idle H100s burn $3+/hr." On-demand H100 prices vary widely by provider and have fallen since 2024; plan with $2–4 per GPU-hour on-demand and less on spot or commitments, and check current prices. The point stands: an idle 8-GPU node costs hundreds of dollars a day, so scheduling is a FinOps job.

PRACTICE, DONE RIGHT

Build a queue-based autoscaler with graceful drain.

1. Requests go into a queue (Redis Streams or similar). Workers running vLLM pull from it.
2. KEDA scales the worker deployment on queue depth and oldest-message age. Allow scale to zero, plus a configurable warm minimum.
3. Nodes come from a spot node pool. On reclaim notice: mark the pod unready, stop pulling, finish in-flight requests, exit.
4. Measure cold-start time end to end, then shrink it (pre-pulled images, local weight cache, faster loader). Record time-to-scale under a traffic spike and dollars spent vs a fixed-size fleet over a simulated day.

Done means a cost-vs-SLO-violation chart comparing fixed, reactive and warm-buffer policies.

READ MORE

- NVIDIA GPU Operator and MIG user guides. Kubernetes DRA docs.
- KEDA scalers documentation; KubeRay and Ray Serve autoscaling docs.
- vLLM production-stack and llm-d deployment guides.

STAGE 11

AI gateways, routing & observability

The gateway decides which model handles a request; observability tells you whether the answer came back fast, correct and cheap.

Time: 2–3 weeks | **Hardware:** one GPU for a local model + API access | **Software:** LiteLLM or Envoy AI Gateway, Prometheus, Grafana, OpenTelemetry, DCGM exporter

The SLOs that matter

Metric	What it measures	Who cares
TTFT	Request arrival → first token	Interactive users; dominated by queueing + prefill
ITL / TPOT	Time between tokens	Streaming UX; humans read ~5–10 tokens/s
E2E latency	Arrival → last token	Agents and batch pipelines
Goodput	Requests/s that meet all SLOs	Capacity planning. The best single number.
Cost per 1M tokens	$/hr ÷ tokens/hr (input and output separately)	Finance, product

Always report percentiles (p50, p95, p99), never just means.

Routing

- **Rule-based**: by endpoint, user tier, prompt length. Boring and very effective.
- **Classifier-based / semantic**: a small model or embedding classifier predicts whether a cheap model will do. RouteLLM is a well-known open framework and paper on this.
- **Cascade**: try the cheap model, verify confidence or validate structured output, and escalate on failure.

The router itself must be cheap: a few milliseconds and a tiny fraction of the cost it saves.

GPU observability with DCGM

The commonly watched "GPU utilization" metric (`DCGM_FI_DEV_GPU_UTIL`) only says whether *any* kernel was running in the sample window. A GPU running one tiny kernel all the time shows 100%. Better signals:

- `DCGM_FI_PROF_SM_ACTIVE` and `DCGM_FI_PROF_SM_OCCUPANCY`: how much of the chip is actually busy.
- `DCGM_FI_PROF_PIPE_TENSOR_ACTIVE`: tensor-core usage.

- `DCGM_FI_PROF_DRAM_ACTIVE` : memory bandwidth usage. During decode, this is the one that should be high.
- Clock throttle (clock event) reasons, temperature and power: this is how you catch a GPU that's silently thermal- or power-throttling.
- XID errors and ECC counters: hardware faults that precede failures.

Tracing with OpenTelemetry

OpenTelemetry has GenAI semantic conventions for LLM spans (model, token counts, finish reason, and so on). Trace a request across gateway → router → engine so you can split TTFT into queueing, routing, prefill and network time. Be careful about logging prompt and response content: it's often sensitive, so make it opt-in and redacted.

MYTH CHECK

"Route by prompt complexity." Prompt length and keywords are weak proxies for difficulty. A short question can need deep reasoning. Routers must be evaluated like models: measure quality on held-out traffic at each cost point, not just the share of traffic that went to the cheap model.

PRACTICE, DONE RIGHT

1. Put a gateway in front of a local quantized 8B model (vLLM) and a frontier API.
2. Build a router: start with rules, then add a small classifier trained on ~2,000 labeled prompts (label = "did the small model's answer match the frontier answer?").
3. Sweep the routing threshold and plot **quality vs cost**. Pick an operating point.
4. Wire Prometheus + Grafana with engine metrics, DCGM metrics and OTel traces. Deliberately throttle a GPU (lower its power limit) and confirm your dashboard catches it.

Done means a quality-vs-cost curve and a dashboard that would page you for the right reasons.

READ MORE

- Ong et al., *RouteLLM*.
- NVIDIA DCGM field identifiers documentation.
- OpenTelemetry semantic conventions for generative AI.

STAGE 12

Public benchmarks & teardowns

The capstone. A rigorous, reproducible public teardown shows everything from the previous eleven stages in one piece of work.

Time: 3–4 weeks | **Hardware:** the hardware you claim results for, reserved for long enough | **Software:** vLLM, SGLang, TensorRT-LLM; vllm bench / sglang bench_serving / GenAI-Perf

What makes a benchmark credible

1. **Pin everything.** Engine versions (commit hashes), driver, CUDA, model revision, quantization recipe, every flag. Publish the exact launch commands.
2. **Realistic traffic.** Real-ish input/output length distributions (e.g. from a public chat dataset), Poisson or bursty arrivals, not "1,000 identical 128-token prompts."
3. **Sweep load, don't pick one point.** Plot **throughput vs latency** curves as request rate rises. The Pareto frontier is the result; a single number is marketing.
4. **Fair tuning.** Give each engine a comparable tuning effort, and say how much you did. Ask the maintainers to review your configs before publishing; they usually help.
5. **Check quality.** Confirm each engine's outputs match a reference on an eval set. A fast engine with a broken sampling path is not faster.
6. **Repeat and report variance.** Warm-up runs, several repetitions, error bars.
7. **Translate to cost.** $/1M output tokens at a stated SLO is what readers actually want.

A teardown outline that works

```
1. TL;DR: one chart, three sentences
2. Setup: hardware, software versions, model, quantization
3. Workloads: chat, long-context RAG, code; length distributions shown
4. Results: latency-vs-throughput curves per workload, goodput at SLO
5. Why: profiler evidence for the biggest differences
6. Cost: $/1M tokens at SLO, per engine
7. Caveats: what you didn't test, what might change next release
8. Reproduce: repo link with scripts, configs, raw data
```

Section 5 is what separates a teardown from a leaderboard. Anyone can run a benchmark script. Explaining *why* engine A wins on long prompts (chunked-prefill policy, attention kernel, scheduler) is what demonstrates expertise.

MYTH CHECK

"Public proof gets you hired instantly by top AI labs." It helps a lot, especially combined with open-source contributions to the engines themselves. "Instantly" is the hype talking. Merged PRs to vLLM, SGLang, Triton or FlashInfer are often an even stronger signal than a blog post.

PRACTICE, DONE RIGHT

Pick **one** model (e.g. Llama 3.1 70B FP8) on **one** hardware config (e.g. 4×H100) and three workloads. Compare vLLM, SGLang and TensorRT-LLM (and optionally Dynamo or llm-d as a deployment layer). Publish the repo with one-command reproduction, plus the write-up. Then send it to the engine maintainers and incorporate their feedback in a v2.

CHAPTER 13

Beyond the roadmap

Five topics the original 12 stages skip. Each comes up constantly in real production work and in interviews.

Topic	Why it matters	Builds on
13.1 Multi-LoRA serving	Hundreds of fine-tuned variants on one fleet instead of one fleet per model	Stages 3, 4, 6
13.2 Structured output	Agents and APIs need valid JSON; the constraint has a real speed cost	Stages 3, 6
13.3 Non-NVIDIA hardware	Supply, price and memory capacity increasingly favour alternatives	Stages 1, 2, 5
13.4 Offline & batch inference	A large share of tokens have no human waiting; different objective entirely	Stages 3, 10
13.5 Reliability & multi-tenancy	Failures, overload, fairness and cross-tenant leaks	Stages 4, 10, 11

13.1 Multi-LoRA serving

LoRA fine-tuning adds a pair of small low-rank matrices next to selected weight matrices instead of changing the weights. An adapter of rank 16 on an 8B model is typically tens of MB, versus 16 GB for the base model. That size gap is the whole opportunity: keep **one copy of the base model** in GPU memory and serve many customers' or tasks' adapters on top of it.

The hard part is batching. A batch might contain requests for 30 different adapters. You can't merge each adapter into the weights, so the shared base GEMM runs once for everyone, and then a special kernel applies each request's own low-rank update. The Punica project introduced the SGMV kernel (segmented gather matrix-vector) for exactly this; S-LoRA added paging of adapters between CPU and GPU memory, much like PagedAttention does for KV blocks. vLLM and SGLang both support this natively (in vLLM: `--enable-lora`, `--max-loras`, `--max-lora-rank`).

> **NAPKIN MATH: ONE FLEET VS MANY**
>
> 100 fine-tuned 8B models served separately: 100 × 16 GB = 1.6 TB of weights, at least 20 H100s even if most models are idle. As adapters: 16 GB base + 100 × ~50 MB = ~21 GB, on one GPU, with the rest of memory free for KV cache. The decode cost of the adapter math is a few percent at moderate ranks.

What to watch: adapter load latency on a cache miss (cold adapters come from CPU or disk), the cap on adapters per batch, higher ranks costing more, and routing. Adapter affinity is the same problem as prefix affinity in Stage 4: send requests for the same adapter to the replica that already

has it loaded.

> **PRACTICE**
>
> Serve an 8B base with 50 adapters (train a few cheaply, clone the rest with random weights for load testing). Use Zipf-distributed adapter popularity. Measure throughput and TTFT as the number of distinct adapters per batch rises, and the TTFT penalty on an adapter cache miss. Compare against a single merged model at the same load.

13.2 Structured output and constrained decoding

When an API promises "the output will match this JSON schema," the engine enforces it during decoding. The schema (or a regex or grammar) is compiled into a state machine. At each step, the engine computes which tokens are legal in the current state and masks every other token's logit to $-\infty$ before sampling. The model literally cannot produce invalid output.

Libraries: **XGrammar** (the default backend in vLLM and SGLang today), **llguidance**, and **Outlines**. The tricky part is that grammars are written in characters but models emit tokens, which may span several grammar symbols. Good libraries precompute which tokens are valid for each state so the per-step cost stays small.

Where the cost comes from

- **Mask computation** runs on the CPU at every step for every constrained sequence. At large batch sizes, or with complex grammars, this can become the step's critical path and leave the GPU waiting. Good engines overlap mask computation with the GPU forward pass.
- **Grammar compilation** for a new schema can take noticeable time. Cache compiled grammars by schema.
- **The upside**: when the grammar forces the next several tokens (e.g. `{"name": "`), the engine can skip the model entirely for those. SGLang calls this jump-forward decoding.

> **QUALITY TRAP**
>
> Forcing a model straight into a rigid schema can hurt answer quality, especially for reasoning-heavy tasks. A common pattern is to let the model reason freely (or in a `reasoning` field placed first in the schema) and constrain only the final answer fields. Measure task accuracy with and without constraints, not just validity rate.

> **PRACTICE**
>
> Benchmark one model with no constraint, a simple JSON schema, and a deeply nested schema, at batch sizes 1 to 128. Measure tokens/s and per-step CPU time, and check in Nsight Systems whether the GPU idles waiting for masks. Then run a reasoning eval with and without a schema to see the quality effect.

13.3 Non-NVIDIA hardware

Everything in Stage 2 is hardware-agnostic: bytes, FLOPs, rooflines. What changes across vendors is memory capacity, bandwidth, interconnect, and above all **software maturity**.

Platform	Strengths	Software stack	Watch out for
AMD Instinct (MI300X 192 GB, MI325X 256 GB, MI355X 288 GB)	More HBM per GPU than NVIDIA equivalents: big models on fewer GPUs, more KV room	ROCm, HIP, RCCL. vLLM, SGLang and Triton all support it	Kernel coverage and tuning lag NVIDIA for new features; validate your exact model and features
Google TPU (v5e, v6e Trillium, Ironwood)	Strong price-performance at scale, large pods with fast interconnect	JAX / XLA; vLLM TPU backend	Different programming model; static shapes; only on Google Cloud
AWS Trainium / Inferentia	Cost on AWS, availability	Neuron SDK; vLLM integration	Model support and compile times; AWS-only
Specialised (Groq, Cerebras, SambaNova)	Very high single-stream tokens/s via on-chip SRAM	Mostly consumed as an API	Mostly API access; limited control over models

> **NAPKIN MATH: MEMORY CAPACITY CHANGES THE DESIGN**
>
> Llama 3 70B in BF16 (140 GB) fits on a **single** 192 GB MI300X with ~40 GB left for KV cache. On H100s it needs two GPUs and tensor parallelism. One GPU means no all-reduce traffic at all. Capacity, not just FLOPs, decides the architecture.

The portable skills: Triton (runs on NVIDIA and AMD), the vLLM/SGLang abstractions, and roofline reasoning. Write your benchmark harness to be hardware-agnostic from day one, and a cross-vendor teardown becomes an easy Stage 12 extension that relatively few people publish carefully.

> **PRACTICE**
>
> Rent an MI300X for a day. Run your Stage 3 workload with the same vLLM version. Compare goodput per dollar against H100, and note every feature or kernel that didn't work or needed a workaround. That list is as valuable as the numbers.

13.4 Offline and batch inference

Evaluations, synthetic data generation, document processing, embeddings over a corpus, nightly classification: nobody is watching a cursor blink. The objective flips from **latency under an SLO** to **tokens per dollar, finished by a deadline**.

What changes

- **Batch as large as memory allows.** Push `max_num_seqs` and the token budget way up. Decode climbs toward the compute roof (Stage 2), which is exactly what interactive serving can't afford.
- **Order the work.** Group requests sharing a prefix so prefix caching hits; sort by length to reduce padding and tail effects.
- **Use the offline API directly.** vLLM's `LLM` class (no HTTP server), or Ray Data for multi-node pipelines with streaming input and output.
- **Spot everything, checkpoint progress.** Write results incrementally and make the job resumable, so a reclaimed node costs minutes, not hours.
- **Quantize more aggressively** if the quality eval allows, since cost is the only goal.
- **Compare with provider batch APIs.** Major API providers offer asynchronous batch endpoints at a large discount (commonly around 50%). Self-hosting has to beat that.

> **PRACTICE**
>
> Process 1M prompts (e.g. summarise a public dataset) with an 8B model. Start with naive defaults, then apply each technique above one at a time and record $/1M output tokens after each. Kill a node mid-run and verify the job resumes correctly.

13.5 Reliability and multi-tenancy

GPUs fail, and in unhelpful ways

- **Hard faults**: XID errors, uncorrectable ECC errors, NVLink errors, GPUs falling off the bus. DCGM reports them (Stage 11); automate cordon-and-drain.
- **Hangs**: a stuck NCCL collective freezes every GPU in a tensor-parallel group. Set collective timeouts and a watchdog that restarts the whole group.
- **Silent degradation**: thermal throttling or one slow GPU in a TP group drags the whole group down to its speed.
- **Honest health checks**: an HTTP 200 from the server process proves nothing. Run a tiny real generation as the readiness probe.

Overload and fairness

- **Admission control**: when KV cache is nearly full, the engine starts *preempting* sequences (recomputing or swapping their KV), and throughput collapses. Shed or queue load at the gateway before that point, based on KV usage and queue depth.
- **Cancel on disconnect**: when a client goes away, abort the request and free its KV blocks immediately. Otherwise abandoned requests keep burning GPU time.
- **Retries are expensive**: a retried 4K-token request repeats the whole prefill. Use retry budgets and backoff to avoid retry storms during incidents.

- **Fairness in tokens, not requests**: one tenant sending 32K-token prompts costs far more than one sending 200-token chats. Rate-limit and fair-queue by tokens; use priority scheduling for interactive vs batch traffic.

Isolation: the prefix-cache side channel

Shared prefix caching creates a **timing side channel**: if a request's TTFT is suspiciously fast, its prefix was already cached, meaning someone else recently sent the same text. Researchers have shown this can leak information about other users' prompts. Mitigation: scope the cache per tenant by mixing a tenant-specific salt into block hashes (vLLM supports a per-request cache salt), and share caches only for public content like your own system prompts.

PRACTICE

Chaos-test your Stage 10 cluster: kill a GPU process mid-generation, inject a network partition inside a TP group, and flood it with one tenant's long prompts. For each, record what users saw and how long recovery took. Then demonstrate the prefix-cache timing leak between two tenants, turn on per-tenant salting, and show the leak is gone.

READ MORE

- Chen et al., *Punica*; Sheng et al., *S-LoRA*.
- Dong et al., *XGrammar*; vLLM and SGLang structured-output docs.
- AMD ROCm documentation for vLLM; the vLLM TPU backend docs.
- vLLM offline inference docs; Ray Data LLM batch inference docs.
- Research on prompt leakage via KV-cache sharing timing; vLLM docs on cache salting.

APPENDIX A

A compressed 6-month plan

The original roadmap is linear. With 20 years of systems experience, you can run stages in parallel and skip what you already know.

Month	Stages	Deliverable
1	1 + 2 together	SGEMM ladder with profiler notes; decode-vs-batch roofline plot on an 8B model.
2	6 (Triton) + finish 2	Fused RMSNorm + residual kernel at 80%+ of peak bandwidth; Nsight trace showing CUDA-graph gap removal.
3	3 + 5	Chunked-prefill goodput study on 70B; FP8 vs INT4 vs BF16 quality/latency table.
4	4 + 8	Cache-aware router with spillover; speculative-decoding speedup-vs-batch chart.
5	7 + 9 + 10 (light)	Parallelism layout study on one node; one disaggregated deployment; KEDA autoscaler with spot drain.
6	11 + 12	Gateway with quality-vs-cost routing; the public teardown.

Budget

Most of months 1–2 runs on one GPU: a local card or a single rented H100 for a few dozen hours. Months 3–6 need multi-GPU nodes in bursts. Rent by the hour, script every experiment so it runs unattended, and tear down immediately. A realistic total is a few hundred to low thousands of dollars in GPU time, depending on prices and how disciplined you are with shutdowns. Reserve the 8-GPU and 2-node experiments for a few focused sessions each.

Working habits that compound

- **Predict before you measure.** Write the napkin-math estimate first. The gap between prediction and measurement is where learning happens.
- **Keep a lab notebook repo.** Every experiment: config, command, raw results, plot, three-line conclusion. It becomes your Stage 12 material.
- **Read engine source, not just docs.** vLLM's scheduler and block manager, SGLang's radix cache, and Triton kernels in FlashInfer are all very readable for an experienced engineer.
- **Contribute upstream early.** A small fix to vLLM or SGLang in month 2 gets you into the community and the code review loop.

- **Use AI assistants for scaffolding, not for numbers.** Let them write benchmark harnesses and plotting code; verify every performance claim yourself.

APPENDIX B

Cheat sheets

Hardware numbers (approximate peak, dense)

GPU	Memory	Bandwidth	BF16 TFLOP/s	FP8 TFLOP/s	Ridge (BF16)
A100 80GB SXM	80 GB	2.0 TB/s	312	n/a	~156
H100 SXM	80 GB	3.35 TB/s	~989	~1,979	~295
H200 SXM	141 GB	4.8 TB/s	~989	~1,979	~206
B200	~180–192 GB	~8 TB/s	~2,250	~4,500	~280

Vendor figures vary by SKU, power limit and configuration. Sustained numbers are typically 60–85% of peak. Use these for estimates only.

Formulas to know by heart

Quantity	Formula
Weight memory	params × bytes per param (BF16 = 2, FP8 = 1, INT4 ≈ 0.5 + scales)
KV bytes per token	2 × layers × kv_heads × head_dim × bytes
Decode speed limit (batch 1)	bandwidth ÷ (weight bytes + KV bytes read per step)
FLOPs per token (forward)	≈ 2 × params (plus attention, which grows with context)
Prefill time (compute-bound)	≈ 2 × params × prompt tokens ÷ (achieved FLOP/s)
Decode arithmetic intensity	≈ batch size (FLOPs/byte, BF16 weights)
Ridge point	peak FLOP/s ÷ bandwidth
Speculative tokens per pass	$(1 - \alpha^{k+1}) \div (1 - \alpha)$
Cost per 1M tokens	($/hour for the GPUs) ÷ (tokens per hour) × 10^6

WORKED EXAMPLE: PREFILL TIME

Llama 3 70B, 4,000-token prompt: $2 \times 70 \times 10^9 \times 4{,}000 = 5.6 \times 10^{14}$ FLOPs. On 4×H100 at ~50% of BF16 peak (~2,000 TFLOP/s total): ~0.28 s. That's your TTFT floor before queueing.

Glossary

All-reduce	Collective that sums tensors across GPUs and gives every GPU the result. Core of tensor parallelism.
All-to-all	Every GPU sends a distinct chunk to every other GPU. Core of expert parallelism.
Chunked prefill	Splitting long prompts into pieces processed alongside decode steps.
Continuous batching	Adding and removing requests from the running batch at every step.
CUDA graph	A recorded sequence of kernel launches replayed with one call.
Disaggregation	Running prefill and decode on separate GPU pools, transferring KV between them.
GQA / MLA	Attention variants that shrink the KV cache (shared KV heads; latent compression).
Goodput	Throughput counting only requests that met their SLOs.
ITL / TPOT	Inter-token latency / time per output token.
MIG	Hardware partitioning of one GPU into isolated instances.
MoE	Mixture of Experts: only a few expert sub-networks run per token.
Occupancy	Resident warps on an SM relative to the maximum.
PagedAttention	KV cache stored in fixed-size blocks with a per-sequence block table.
Roofline	Model bounding performance by compute peak or bandwidth × intensity.
SM	Streaming Multiprocessor, the GPU's basic compute unit.
TTFT	Time to first token.
Warp	32 threads that execute in lockstep.

The short reading list

If you only read ten things:

1. Horace He, *Making Deep Learning Go Brrrr From First Principles*
2. Google DeepMind, *How to Scale Your Model*
3. Simon Boehm, *How to Optimize a CUDA Matmul Kernel*
4. Kwon et al., *PagedAttention / vLLM* (SOSP 2023)
5. Agrawal et al., *Sarathi-Serve* (OSDI 2024)
6. Dao et al., *FlashAttention* (1, 2 and 3)
7. Lin et al., *AWQ*, and Frantar et al., *GPTQ*
8. Leviathan et al., *Fast Inference via Speculative Decoding*, plus the EAGLE papers

9. Zhong et al., *DistServe*, and Qin et al., *Mooncake*
10. The vLLM and SGLang source code, starting from the scheduler

End of guide. Wrappers may be a commodity, but the physics doesn't change: count the bytes, place the workload on the roofline, and measure before you believe.